

Conv-ChArT — the attention gate

Read from the live module; every constant below is the one in dcc/model.py

Formulation

Additive attention gate (Oktay et al., *Attention U-Net*, eq. 1–2), applied to the decoder’s H/4 skip connection and conditioned on the post-attention bottleneck. Writing s for the encoder skip and g for the gating signal:

$$\alpha = \sigma\left(\psi\left(\text{ReLU}(W_x * s_{\downarrow 2} + W_g * g)\right)\right), \quad \tilde{s} = s \odot \text{up}_{\text{bilinear}}(\alpha)$$

- W_x : 1×1 conv, stride 2, **no bias** — $128 \rightarrow 64$ channels. The stride is what brings the skip down to the gating signal’s resolution.
- W_g : 1×1 conv — $256 \rightarrow 64$ channels.
- ψ : 1×1 conv — $64 \rightarrow 1$ channel. Collapses to a **single spatial map**, not a per-channel mask.
- σ : logistic sigmoid, so $\alpha \in (0, 1)$ per spatial position.
- up: bilinear, **align_corners=False**, back to the skip’s resolution.
- $\odot$: elementwise, broadcast across all 128 skip channels.

Total: **24,705 parameters** (0.5% of the 4,698,034-parameter detector). The gate is nearly free; what it costs is a design commitment, not compute.

Pass-through initialisation

ψ ’s **weight is zeroed** and its **bias set to +3.0**, so at step 0

$$\alpha \equiv \sigma(3.0) = 0.9526 \quad \text{exactly constant, everywhere.}$$

Verified on the constructed module: $\max |\psi_W| = 0.0$, $\psi_b = 3.0$.

Why zero the weight rather than just bias it high. A large bias alone (say +5, giving $\alpha \approx 0.993$) would leave α *technically* input-dependent from the first step — the network would begin by fighting a small, arbitrary gating signal it never asked for. Zeroing W_ψ makes the gate a **true no-op** at initialisation: the skip passes through scaled by a constant, and training decides if and when gating should begin. The Oktay reference implementation’s TORR variant uses exactly $\psi_b = +3.0$; the zeroed weight is our strengthening of it.

Where it is applied, and where it is not

At `attend_div = 8` the gate sits on the **H/4 skip only**. The H/2 and full-resolution skips are ungated by design.

The rule is: gate only where a wrong veto can be **undone downstream**. The H/4 skip feeds the class head, and the Stage-3 lattice gate plus recovery pass can repair a corner whose identity was lost. The full-resolution skip feeds the heatmap head, where a suppressed corner is simply gone — there is no recovery mechanism for a detection that was never made. Gating the high-resolution skips would put an unrecoverable failure mode behind a learned sigmoid, for no compensating benefit.

Ablation lever

`gates_enabled: false` (`configs/abl_gates_off.yaml`) bypasses the gate — $\tilde{s} = s$ — but still **constructs** the module, so the `state_dict` shape is unchanged and a gated checkpoint stays loadable. Ablation by bypass, not deletion, keeps parameter counts comparable between arms. (Contrast `abl_nodilate.yaml`, which *deletes*, because there the parameter saving is the hypothesis.)

What it actually learned (40 frames, trained weights)

	median α
inside the board region	0.444
outside it	0.559
whole map	0.553
ratio inside/outside	0.79

α has moved well away from its pass-through initialisation of 0.9526, so the gate is doing something rather than passing the skip through untouched. It is **content-conditioned, not a fixed spatial prior** — swapping the conditioning signal g for another frame’s changes α materially.

Two cautions worth carrying into any claim about this. The inside/outside ratio is **not stable in sign across frames** — it has been measured both above and below 1 on different samples — so the honest statement is “content-conditioned”, not “the gate suppresses/passes the board”. And α is a per-pixel scalar broadcast over all 128 skip channels, so it selects *where*, never *which feature*.